사용 설명서 & 개발 정리

Screenshot to Text

F12 한 번으로 화면 속 코드를 텍스트로.
한글·영어 OCR 내장, API 키 없이 오프라인 동작.

F12

Windows 11 · Python 3.14 · Tkinter

RapidOCR + 한국어 인식 모델

v1.3.0 · 2026-11-04

목차

1. 주요 기능
2. 설치
3. 사용법
4. 동작 원리
5. 설정 (`config.json`)
6. OCR 백엔드
7. 저장 위치
8. 파일 구조
9. 구현 노트 (개발 정리)
10. 검증한 내용
11. 문제 해결
12. 알려진 한계
13. 개발 환경
14. 라이선스 / 출처
15. 변경 이력

F12 를 누를 때만 동작하는 코드 스크린샷 → 텍스트 도구입니다. 평소에는 트레이에서 조용히 대기하고, **F12를 눌렀을 때만** 화면을 캡처합니다.

```
F12  →  마우스로 코드 영역 드래그  →  코드가 Windows 클립보드에 복사됨
```

> 실행 중이라도 F12를 누르기 전에는 아무 것도 하지 않습니다. 캡처는 오직 F12 입력으로만 시작됩니다.

한글 주석/문자열도 인식됩니다. **한국어 OCR 모델이 내장**되어 있어 API 키 없이 오프라인으로 동작합니다.

---

1. 주요 기능

기능	설명
F12 전역 단축키	어느 앱에서든 F12만 누르면 캡처 시작
영역 선택 오버레이	화면이 어두워지고, 드래그한 영역만 밝게 보임
한글 + 영어 OCR	내장 한국어 모델로 주석/문자열/코드 동시 인식
들여쓰기 복원	단어 박스 좌표로 들여쓰기·띄어쓰기·빈 줄 복원
줄번호 자동 제거	왼쪽에 붙은 줄번호를 감지해 제거
클립보드 자동 복사	Win32 클립보드 사용 → `Win+V` 기록에도 저장
포커스 안전	캡처 후 원래 작업하던 창으로 포커스 복구
키 입력 무간섭	등록된 키만 가로챔 → Alt+Tab/Ctrl/Alt/Win 영향 없음
트레이 상주	콘솔 창 없이 백그라운드 대기
결과 창 (선택)	필요하면 창에서 수정·저장·다시 캡처 가능
OCR 백엔드 4종	RapidOCR(기본) · Windows OCR · Vision API · Tesseract
단일 실행	중복 실행 시 안내 후 종료

---

2. 설치

자동 설치

1. Python 3.10 이상 설치 (설치 시 **Add python.exe to PATH** 체크)

2. 이 폴더의 `install.bat` 더블클릭 - `.venv` 가상환경 생성 - 필수 패키지 설치 (최초 1~3분) - 한국어 OCR 모델 다운로드 (`models/korean`) - `config.json` 생성
3. `run.bat` 더블클릭 → 트레이에 파란 아이콘 생성
4. **F12** → 영역 드래그 → `Ctrl+V` 로 붙여넣기

수동 설치

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
.venv\Scripts\python.exe tools\fetch_models.py
python app.py
```

---

3. 사용법

동작	설명
F12	캡처 시작 (화면이 어두워지고 십자 커서)
드래그	코드 영역 선택 → 놓는 순간 인식 후 클립보드에 복사
Esc / 우클릭	캡처 취소
Ctrl+Alt+Q	프로그램 종료 (트레이 메뉴의 `종료` 도 가능)

기본 동작 (결과 창 없음)

- 선택을 놓으면 **곧바로 클립보드로 복사**되고, 커서 옆에 `복사됨 · N줄` 배지가 1~2초 표시됩니다.
- 인식 중에는 `코드 인식 중…` 배지가 표시됩니다.
- 텍스트를 못 찾으면 `텍스트를 찾지 못했습니다` 배지가 뜨고 클립보드는 그대로 둡니다.
- 배지는 `WS_EX_NOACTIVATE` 창이라 **포커스를 빼앗지 않습니다.**
- 캡처/취소가 끝나면 **F12를 누르기 전에 쓰던 창으로 포커스가 돌아갑니다.**

결과 창으로 보고 싶을 때

`config.json` 에서 `behavior.show_result_window` 를 `true` 로 바꾸면:

- 인식 결과가 창으로 뜨고, 텍스트를 직접 수정할 수 있습니다.
- **복사**(`Ctrl+C`) · **저장**(`Ctrl+S`) · **원본**(PNG 열기) · **다시 캡처** 버튼 제공
- 창을 닫으면 역시 원래 창으로 포커스가 돌아갑니다.

트레이 메뉴

- `캡처 시작 (F12)`

- `마지막 결과 보기` — 방금 복사한 결과를 창에서 다시 확인
- `캡처 폴더 열기` / `설정 파일 열기`
- `종료`

---

4. 동작 원리

```
[F12 입력]
   │
   ├─ 전역 단축키 훅: F12만 감지해서 앱에 알림 (다른 키는 그대로 통과)
   │
[화면 스냅샷]  mss로 전체 가상 화면을 한 번에 캡처
   │
[선택 오버레이]  모니터별 dim 창 + 드래그 영역만 원본으로 표시
   │                ESC/우클릭 → 취소, 놓으면 확정
[영역 자르기]  PNG로 저장(captures/), 텍스트 저장 옵션
   │
[OCR]  워커 스레드에서 실행 (UI 멈춤 없음)
   │     RapidOCR + 한국어 모델 (또는 선택한 백엔드)
   │     단어 박스 → 줄 묶기 → 들여쓰기/띄어쓰기/빈 줄 복원 → 줄번호 제거
   │
[결과]  Win32 클립보드 복사 → 배지 표시 (또는 결과 창)
   │
[포커스 복구]  캡처 전 창 핸들로 SetForegroundWindow
```

---

5. 설정 (`config.json`)

첫 실행 시 자동 생성됩니다. 값을 바꾼 뒤에는 트레이에서 종료 후 `run.bat` 으로 다시 실행하세요.

최상위

키	기본값	설명
`hotkey`	`"f12"`	캡처 단축키 (`"ctrl+alt+s"` 등으로 변경 가능)
`quit_hotkey`	`"ctrl+alt+q"`	종료 단축키
`tray`	`true`	트레이 아이콘 표시

`ocr`

키	기본값	설명
`backend`	`"auto"`	`auto` / `rapid` / `winocr` / `vision` / `tesseract`

키	기본값	설명
`rec_model`	`"auto"`	`auto` / `korean` / `default` (한국어 모델 선택)
`languages`	`["ko-KR","en-US"]`	Windows OCR/Tesseract 언어 우선순위
`upscale`	`"auto"`	작은 이미지 확대 배율 (`"auto"` 또는 숫자)
`strip_line_numbers`	`true`	줄번호 자동 제거
`preserve_indent`	`true`	들여쓰기 복원
`tab_width`	`4`	탭 → 공백 변환 폭
`min_confidence`	`0.25`	RapidOCR 최소 신뢰도
`tesseract_path`	`""`	Tesseract 실행 파일 경로 (비우면 자동 탐색)
`tesseract_psm`	`6`	Tesseract 페이지 세그먼테이션 모드

`behavior`

키	기본값	설명
`copy_to_clipboard`	`true`	인식 직후 자동 복사
`save_png`	`true`	캡처 원본 PNG 저장
`save_text`	`false`	결과 텍스트도 자동 저장
`output_dir`	`"captures"`	저장 폴더 (상대/절대 경로)
`dim`	`0.55`	선택 영역 바깥 어둡게 하는 정도 (0 ~ 0.85)
`popup_near_selection`	`true`	결과 창을 선택 영역 근처에 배치
`show_result_window`	`false`	`true` 면 결과 창 표시
`show_toast`	`true`	복사/오류 배지 표시

`vision`

키	기본값	설명
`base_url`	`https://api.openai.com/v1`	OpenAI 호환 엔드포인트
`model`	`gpt-4o-mini`	Vision 모델 이름
`api_key`	`""`	API 키 (비우면 환경 변수 사용)
`api_key_env`	`OPENAI_API_KEY`	키 환경 변수 이름

키	기본값	설명
`timeout`	`90`	HTTP 타임아웃(초)
`max_tokens`	`8192`	최대 응답 토큰
`extra_body`	`{}`	provider 별 추가 파라미터
`prompt`	(내장)	코드 추출 지시문

---

6. OCR 백엔드

`ocr.backend` 가 `auto` 이면 **API 키가 설정된 경우 `vision` 을 먼저** 쓰고, 그 외에는 `rapid → winocr → tesseract` 순서로 사용 가능한 첫 엔진을 씁니다.

backend	엔진	정확도	준비
`rapid`	RapidOCR (오프라인 ONNX, 한국어+영어)	보통 (기본값)	`install.bat` 으로 자동 설치
`winocr`	Windows 내장 OCR	낮음~보통	Windows 언어 팩 필요
`vision`	Vision LLM API	가장 좋음	API 키 필요
`tesseract`	Tesseract CLI	보통	별도 설치 + `tesseract_path`

- 기본 `rapid` 는 **한국어 인식 모델**(`models/korean`)로 한글 + 영어를 함께 읽습니다.
- 코드 기호(`_` , `->` , 따옴표)까지 완벽해야 하면 `vision` 이 가장 정확합니다.
- API 키가 있고 `auto` 라면 `vision` 이 자동 선택됩니다.

한국어 모델

`models/korean/rec.onnx` , `models/korean/dict.txt` 가 있으면 한국어를 인식합니다. `install.bat` 이 자동으로 내려받고, 이미 있으면 건너뜁니다.

```
:: 직접 다시 받을 때
.venv\Scripts\python.exe tools\fetch_models.py
```

Vision API 예시

OpenAI:

```
"vision": {
  "base_url": "https://api.openai.com/v1",
  "model": "gpt-4o-mini",
  "api_key": "sk-..."
}
```

OpenRouter:

```
"base_url": "https://openrouter.ai/api/v1",
"model": "google/gemini-2.5-flash"
```

Google Gemini (OpenAI 호환):

```
"base_url": "https://generativelanguage.googleapis.com/v1beta/openai",
"model": "gemini-2.5-flash"
```

로컬 Ollama:

```
"base_url": "http://127.0.0.1:11434/v1",
"model": "qwen2.5vl:7b",
"api_key": "ollama"
```

---

7. 저장 위치

경로	내용
`captures/shot_*.png`	캡처 원본 이미지
`captures/code_*.txt`	인식 결과 (`save_text` = true 일 때)
`app.log`	실행 로그 (문제 발생 시 확인)
`config.json`	사용자 설정

---

8. 파일 구조

```
app.py                  실행 진입점 (단축키, 트레이, 흐름 제어)
config.json             사용자 설정
models/korean/          한국어 인식 모델 (rec.onnx + dict.txt)
tools/fetch_models.py   한국어 모델 다운로드
tools/build_pdf.py      README → PDF 생성 (문서용)
docs/                   이 문서의 PDF/HTML 사본
stc/
  capture.py            mss 화면 캡처
  clipboard.py          Win32 클립보드 복사 (Win+V 기록 호환)
  hotkey.py             전역 단축키 (최소 간섭 키보드 훅)
  winfocus.py           캡처 후 원래 창으로 포커스 복구
  snipper.py            영역 선택 오버레이 (Tk)
  result_window.py      결과 창 UI (설정으로 켜는 옵션)
  toast.py              포커스를 뺏지 않는 복사/오류 알림 배지
  tray.py               트레이 아이콘
  config.py             설정 로딩/저장
  icon.py               트레이/창 아이콘
  theme.py              색상/폰트
  ocr/
    __init__.py         백엔드 선택 및 실행
    rapid.py            RapidOCR 백엔드 (+한국어 모델)
    winocr.py           Windows OCR 백엔드
    vision.py           OpenAI 호환 Vision API 백엔드
    tesseract.py        Tesseract 백엔드
    layout.py           단어 박스 → 코드 텍스트 (들여쓰기/줄번호/띄어쓰기 복원)
install.bat             설치
run.bat                 실행 (콘솔 없이)
run-console.bat         실행 (로그 보기)
```

이 문서는 `docs/Screenshot-to-Text_Guide.pdf` 로도 제공됩니다. 수정 후 다시 만들려면:

```
.venv\Scripts\python.exe tools\build_pdf.py
```

---

9. 구현 노트 (개발 정리)

9.1 화면 캡처와 DPI

- `SetProcessDpiAwarenessContext(PER_MONITOR_AWARE_V2)` 로 물리 픽셀 좌표를 사용합니다.
- `mss` 로 가상 화면 전체(모든 모니터)를 **한 번에** 캡처합니다. Tk 좌표와 1:1로 맞습니다.
- 모니터별 해상도/배율이 달라도 캡처 원본에서 잘라내므로 좌표가 어긋나지 않습니다.

9.2 영역 선택 오버레이

- 모니터마다 `overrideredirect` 최상위 창을 하나씩 생성합니다.

- 배경은 캡처 시점의 화면을 어둡게(`dim`) 만든 이미지 → 드래그 중인 영역만 원본으로 표시.
- 드래그 중 크기(`1234 × 567`), 모서리 핸들, 십자 가이드, 안내 문구 표시.
- 선택은 드래그를 시작한 모니터 안으로 제한됩니다(모니터 경계 클램프).
- 놓는 순간 확정, `Esc`/우클릭은 취소. 오버레이는 **완전히 파괴**되어 화면에 남지 않습니다.

9.3 포커스 처리 (키 입력 먹통 방지)

- F12 직전의 `GetForegroundWindow()` 핸들을 기억합니다.
- 오버레이는 `AttachThreadInput` + `SetForegroundWindow` 로 확실히 포커스를 가져와 Esc가 동작합니다.
- 선택/취소/결과 창 닫기 직후 **원래 창으로 포커스를 복구**합니다.
- 64비트 HWND가 잘리지 않도록 모든 Win32 호출에 argtypes/restype를 명시했습니다.

9.4 전역 단축키 (Alt+Tab 무간섭)

- `keyboard` 라이브러리를 제거했습니다. 이 라이브러리는 모든 키를 훅으로 받아 수정키 상태를 재생/필터링해서 Alt+Tab 같은 조합키가 가끔 꼬였습니다.
- 대신 `WH_KEYBOARD_LL` 최소 훅을 직접 구현했습니다.
- 등록된 키(**F12**, 캡처 중 **Esc**, **Ctrl+Alt+Q**)만 삼키고 나머지는 그대로 통과.
- 수정키 상태를 저장/재생하지 않음.
- 수정키 일치 여부는 `GetAsyncKeyState` 로 그 순간만 확인.
- key-up 을 놓쳐도 2초 뒤 자동 복구되어 키가 영구적으로 막히지 않음.
- Esc는 캡처 중에만 등록되고, 캡처가 끝나면 즉시 해제됩니다.
- 등록된 키는 auto-repeat 시 한 번만 동작하도록 디바운스합니다.

9.5 코드 레이아웃 복원

OCR 결과는 단어 박스 목록입니다. 이를 코드처럼 보이게 재조립합니다.

1. **줄 묶기** — 세로 중심 좌표가 가까운 단어들을 한 줄로 묶음.
2. **띄어쓰기** — 단어 사이 간격을 글자 폭으로 나눠 0/1/2칸 복원.
3. **들여쓰기** — 줄의 x 좌표를 글자 폭으로 나눠 칸 수 계산. 글자 폭은 단어 폭/글자 수의 중앙값을 **격자에 맞춰 미세 보정**합니다.
4. **빈 줄** — 줄 간격(pitch)의 배수로 사라진 빈 줄을 복원.
5. **줄번호 제거** — 왼쪽 열이 3줄 이상 연속 숫자이고 증가하면 줄번호로 판단해 제거.
6. **기호 정규화** — OCR이 자주 내놓는 전각/따옴표/`〈〉` 등을 ASCII로 변환.

9.6 OCR 백엔드

- 공통 인터페이스: `run(image, cfg) -> OcrResult`
- `auto` 선택 시 vision(키 있을 때) → rapid → winocr → tesseract 순으로 시도하고, 실패하면 다음 백엔드로 자동 폴백합니다.
- **한국어 모델**: PaddleOCR 한국어 rec 모델을 ONNX로 변환한 것을 사용합니다. `rapidocr-onnxruntime 1.2.3` 이 `rec_keys_path` 를 읽지 않는 문제는 `TextRecognizer` 초기화를 얇게 보정(shim)해서 해결했습니다.
- 작은 캡처는 자동 업스케일(짧은 변 기준)해서 인식률을 보정합니다.

- Vision 백엔드는 표준 라이브러리(`urllib`)만 사용해 추가 의존성이 없습니다. 응답의 코드 펜스(```)는 자동으로 제거합니다.

9.7 클립보드

- Tk 클립보드는 앱이 종료되면 내용이 사라질 수 있고 Win+V 기록에도 잘 안 남습니다.
- 그래서 Win32 `CF_UNICODETEXT` 로 직접 기록합니다. (CRLF 줄바꿈)
- 실패하면 Tk 클립보드로 폴백합니다.

9.8 그 외

- **단일 실행**: Win32 뮤텍스로 중복 실행 방지.
- **트레이**: pystray, 실패해도 앱은 계속 동작.
- **알림 배지**: `WS_EX_NOACTIVATE | WS_EX_TOOLWINDOW` 로 포커스/Alt+Tab에 영향 없음.
- **로그**: `app.log` (1MB 넘으면 `app.log.1` 로 로테이션).
- **캡처 히스토리**: PNG 파일로 남겨 원본 대조 가능.

---

10. 검증한 내용

개발 중 실제로 확인한 항목입니다.

- **OCR 정확도**: 합성 코드 스크린샷(다크/라이트, 12~22px, 줄번호 포함)으로 RapidOCR / Windows OCR 결과 비교, 한글 주석 포함 샘플 검증
- **한국어 모델**: `안녕하세요 세계`, `설정 파일을 읽습니다` 등 한글 라인 인식 확인
- **오버레이 좌표**: 2모니터(2560×1600, 150% 배율) 환경에서 드래그 이벤트를 시뮬레이션해 선택 Rect(전역 좌표)가 정확히 일치하는지 확인
- **단축키**: SendInput 으로 F12 / Esc / Ctrl+Alt+Q 를 보내 콜백이 정확히 한 번 실행되는지 확인 (수정키 조합 매칭 포함)
- **Vision 백엔드**: 로컬 모의 HTTP 서버로 요청 형식/인증 헤더/코드 펜스 제거 확인
- **포커스**: 캡처 전 창 핸들 복구 로직, 64비트 HWND 유효성 확인

---

11. 문제 해결

- **F12를 눌러도 반응이 없어요**
- 관리자 권한으로 실행 중인 앱 위에서는 키 입력을 감지할 수 없습니다. 이 도구도 관리자 권한으로 실행해 보세요.
- `config.json` 의 `hotkey` 값이 `f12` 인지 확인하세요.
- `app.log` 에 `키보드 훅 시작`, `전역 단축키 등록: f12` 가 있는지 확인하세요.
- **Alt+Tab / Ctrl / Alt / Win 조합이 이상해요**

- 현재 버전은 등록된 키만 가로채므로 다른 조합키에 영향을 주지 않습니다.
- 그래도 이상하면 다른 앱(키보드 유틸, 게임 오버레이 등)의 훅과 충돌일 수 있습니다.
- **브라우저에서 F12(개발자 도구)가 안 열려요**
- 실행 중에는 F12를 캡처 단축키로 사용합니다. 개발자 도구가 필요하면 `hotkey` 를 `"ctrl+alt+s"` 등으로 바꾸세요.
- **한글이 인식되지 않아요**
- `models/korean/rec.onnx` , `models/korean/dict.txt` 존재 여부 확인
- 없으면 `.venv\Scripts\python.exe tools\fetch_models.py` 실행
- `ocr.rec_model` 이 `"auto"` 또는 `"korean"` 인지 확인
- **Win+V 기록에 안 쌓여요**
- `설정 → 시스템 → 클립보드 → 클립보드 기록` 이 켜져 있어야 합니다.
- **인식이 느려요**
- 첫 캡처는 모델 로딩으로 2~4초 걸릴 수 있고, 이후에는 보통 1~2초입니다.
- 캡처를 작게 잡을수록 빨라집니다.
- **들여쓰기가 조금 어긋나요**
- 글꼴이 고정폭(monospace)일 때 가장 정확합니다. 결과 창에서 수정하거나 `vision` 을 사용하세요.
- **결과 창이 안 떠요**
- 기본값이 `show_result_window: false` 라서 정상입니다. 클립보드로 복사됩니다.
- **두 번 실행했어요**
- 이미 실행 중이면 안내창이 뜨고 중복 실행되지 않습니다.
- **여러 모니터**
- 드래그를 시작한 모니터 안에서 선택됩니다. 캡처 자체는 모든 모니터를 포함합니다.

---

12. 알려진 한계

- 로컬 OCR은 기호(`_` , `->` , 따옴표) 인식이 완벽하지 않습니다. 최고 정확도가 필요하면 `vision` .
- 한글 폰트가 비정상적으로 작은 화면(10px 이하)에서는 인식률이 떨어집니다.
- 관리자 권한 앱이 전면에 있으면 이 도구(일반 권한)가 F12를 받지 못합니다.
- `vision` 백엔드는 선택한 이미지를 설정한 API 서버로 전송합니다. (로컬 백엔드는 전송 없음)
- Windows 전용입니다.

---

13. 개발 환경

항목	내용
OS	Windows 11 (10.0.26300)

항목	내용
Python	3.14
GUI	Tkinter (표준 라이브러리)
캡처	mss
이미지	Pillow
OCR	rapidocr-onnxruntime + onnxruntime
Windows OCR	winrt-runtime 계열
트레이	pystray
클립보드/포커스/단축키	ctypes + Win32 API

---

14. 라이선스 / 출처

- 이 프로젝트 코드: 자유롭게 사용/수정 가능.
- 한국어 OCR 모델: PaddleOCR 한국어 인식 모델을 ONNX로 변환한 것 (monkt/paddleocr-onnx, Apache-2.0).
- RapidOCR: Apache-2.0.

---

15. 변경 이력

버전	내용
1.0.0	F12 캡처, 영역 선택 오버레이, 코드 OCR, 클립보드 복사
1.1.0	한국어 OCR 모델 내장, 결과 창/배지, 트레이
1.2.0	포커스 복구, `keyboard` 라이브러리 제거(Alt+Tab 무간섭), 64비트 HWND 수정
1.3.0	이름 변경(Screenshot to Code → Screenshot to Text), README/PDF 문서 정리
